Autonomous and Adaptive Systems - Project Report

Pietro Milo Tasinato
pietromilo.tasinato@studio.unibo.it

Abstract

The goal of this project is to train two actors to solve a classic StagHunt environment. To achieve this, we will need to train two agents to cooperate in the hunt for the stag, so we will implement from scratch the REINFORCE algorithm, and then we will gradually improve on it, in order to improve performance and convergence.

1 Introduction

We begin by analyzing the task environment following the classification introduced by Russel and Norvig [2]: fully observable, multi agent, cooperative, nondeterministic (the deer moves randomly), sequential, dynamic, discrete and unknown (the agent receives only a reward, the knowledge of how that reward is calculated is not included in its knowledge base).

In this specific implementation of the StagHunt game, the environment is made of a grid of cells, the dimension of the grid is variable, but in this project it will remain constant at 5x5, which creates a space of 25 cells. Each cell can contain up to 3 entities. These entities can be of 3 different kinds: a hunters, a stag or a plant. The two agents will take the role of two hunters, they can choose between 5 moves: (up, down, left, right, stay), the stag moves randomly, the plants do not move but once harvested they will be removed from the current cell and moved to another random cell. Regarding the rewards structure, if the two agents cooperate by reaching at the same time the cell in which is situated the stag they both receive the maximum reward. If only one agent reaches the stag, it receive a negative reward; and if one agent reaches a plant, it receives a positive but small reward.

With the knowledge of the reward structure, we can now create a payoff matrix, considering the following rewards: +5 for a successful cooperation, +1 for the agent who defects, -5 for the agent who cooperates when the other agent defects (we assume that if an agent defects, they are going for a plant).

Table 1: Payoff matrix

	Agent 2 cooperates	Agent 2 defects
Agent 1 cooperates	+5, +5	-5, +1
Agent 1 defects	+1, -5	+1, +1

Note that during the implementation and training sessions these values will be altered and we will also alter the reward structure to try to favor more cooperation between the agents, but not enough to transform the reward structure into the one from another game.

2 Implementation

For this project, I chose to implement the Reinforce algorithm. I chose Reinforce instead of another powerful algorithm like DQN because of the multi agent nature of the game. Even though DQN is more stable and usually converges faster than Reinforce, it does not behave well in multi agent

Preprint. Under review.

environments such as StagHunt. Since Reinforce is an on-policy method, even after a very long sequence of episodes where the two agents defect, there will be a small probability for them to cooperate. DQN instead will escape from reciprocal defections less easily. Because of the high variance of Reinforce, we will start by implementing also a baseline to improve convergence and reduce noise. Lastly, if Reinforce will not be enough to solve the environment, we will try with PPO instead of Reinforce.

Preprocessing The gym environment can return the positions of the agents as a list, each list contains a couple of coordinates for each entity in the environment. Then I will use a one hot encoding for each entity in order to precisely identify each cell. This means that the input for the network will be a [5, 25] vector, for a total of 125 sparse values.

2.1 Reinforce with Baseline algorithm

Reinforce is a policy gradient methods, so the policy can be parametrized with one caveat: $\pi(a|s,\theta)$ must be differentiable with respect to its parameters as long as $\nabla pi(a|s,\theta)$ exists and is finite $\forall s \in S$, $a \in A(s)$ and $\theta \in \mathbb{R}^{d'}$; with S the state space, $A(s)$ the action space given a specific state, and θ a vector of weights for the neural network. We then need a way to parametrize our policy to obtain a preferable action given a certain policy, we can use exponential soft-max distribution as such:

$$\pi(a|s,\theta) = \frac{e^{h(s,a,\theta)}}{\sum_b exp(h(s,b,\theta))} \tag{1}$$

Now we introduce the concept of discounted cumulative return generated by step t:

$$G_t = \sum_{k=t+1}^{T} \gamma^{k-t-1} R_k = R_{t+1} + \gamma G_{t+1} \tag{2}$$

with $\gamma \in [0,1]$ being the discount rate, and T the number of steps we will consider (in our case we will consider the full episode). We then need to introduce performance as the value of the start state of the episode, since we want to maximize our expected cumulative reward, maximizing the reward obtained in the first state is the way to assure maximum reward for each consecutive step. We define performance as:

$$J(\theta) = v_{\pi_\theta}(s_0) \tag{3}$$

Where v_{π_θ} is the true value function for π_θ, the policy determined by θ. Directly calculating $J(\theta)$ is much more difficult than it would appear, so we instead need a different solution. Performance is determined by two factors: the actions we take in certain states and the states we visit while following the policy. But usually the effects of those factors are decided by the environment, whose reward function is unknown by the agents. Here we introduce the Policy Gradient Theorem:

$$\nabla J(\theta) \propto \sum_s \mu(s) \sum_a q_\pi(s,a) \nabla \pi(a|s,\theta) \tag{4}$$

with $\mu(s)$ the on-policy distribution under policy π and it describes the normalized frequency with which states are encountered while the agent is selecting actions according to the policy itself; and $q_\pi(s,a)$ the value function for the policy π in the state s while choosing the action a. Given the definition of $\mu(s)$, we can see the right-hand side of the theorem as a sum over states weighted by how often the state occurs under the target policy π, witch is the definition of expected value of a random variable, given that the agent follows π. So the Policy Gradient Theorem becomes:

$$\nabla J(\theta) \propto E_\pi[\sum_a q_\pi(S_t,a) \nabla \pi(a|s,\theta)] \tag{5}$$

Yet since the agent does not know the value function $q_\pi(s,a)$, we need some other algebraic operations to obtain a function usable with Reinforce (the algebraic operations are explained in[1]):

$$\nabla J(\theta) \propto E_\pi[G_t \frac{\nabla \pi(a|S_t,\theta)}{\pi(a|S_t,\theta)}] = E_\pi[G_t \nabla ln(\pi(a|S_t,\theta))] \tag{6}$$

We can now take a look at the proper Reinforce algorithm, starting with stochastic gradient ascent:

$$\theta_{t+1} = \theta_t + \alpha \nabla \hat{J}(\theta_t) = \theta_t + \alpha \sum_a \hat{q}_\pi(S_t, a, \omega) \nabla \pi(a|S_t, \theta) \tag{7}$$

with ω the vector containing the learned weights of the network. By applying the equality in (6), we obtain:

$$\theta_{t+1} = \theta_t + \alpha[G_t \nabla ln(\pi(a|S_t, \theta))] \tag{8}$$

There is one final detail we need to add: the backpropagation algorithm used in neural networks to update the connection weights works by minimizing the loss function, performing stochastic gradient descent, which is the opposite of what we are using. The solution is to simply consider the opposite function to the one the network is computing gradient ascent:

$$\theta_{t+1} = \theta_t - \alpha[G_t \nabla ln(\pi(a|S_t, \theta))] \tag{9}$$

Adding the baseline Due to the high variance, instead of using the plain Reinforce algorithm we will use Reinforce with baseline. The theoretical basis is very similar to Reinforce, we just need to add a $smoothing factor$ while calculating the weight update. This term can be any function, as long as it is independent from a, the action chosen at each state. This requirement of independence from a is necessary in order to guarantee the absence of bias, so that the baseline function does not affect the expected value E_π. So the final weight update function becomes:

$$\theta_{t+1} = \theta_t + \alpha[(G_t - b(S_t)) \nabla ln(\pi(a|S_t, \theta))] \tag{10}$$

In order to achieve the best convergence speed, it is important to choose the baseline appropriately for each state: in some states all actions have high values, and we need a high baseline to differentiate the higher valued actions from the less highly valued ones; in other states all actions will have low values and a low baseline is appropriate. So we could use as baseline function an estimate of the state value: $\hat{v}(S_t, \omega)$, with ω a vector of learned weights used just by the baseline function.

Given the small dimension of the input data and the action space, the agent network is composed by 3 dense layers, 2 of them use a Relu activation function while the last uses a softmax to select the action to use. The baseline network uses 2 dense layer with Relu activation as the agent, but the output is calculated by a linear layer since it needs to be a scalar value. Note that in the image the term "epoch" is used improperly, I use it to indicate a certain number of iterations in the exploration step (in this and in future graphs, i epoch = 400 iterations)

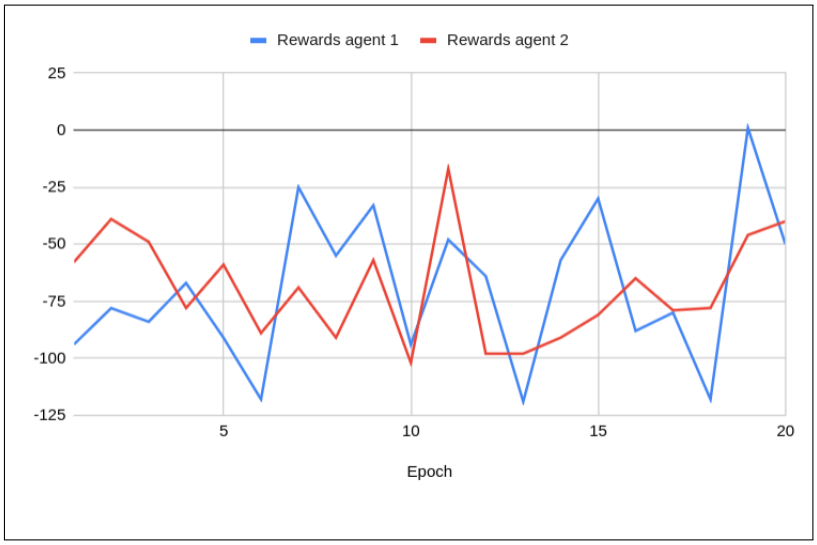


Figure 1: Reinforce with baseline

Looking at the overall behavior of the two agents we clearly see they are inadequate at solving the environment. We can obviously see that the rewards each agent obtained at the end of each episode

did not increased as the episodes went on. We can already see a major difficulty with StagHunt: since this is a cooperative game, the optimal policy for one agents is tied to the policy of the other agent, this causes each training session to try to reach a "moving target" since both policies are being changed. This means that the agent 1 is trying to optimize the expected rewards based on the old policy of agent 2 (the one that agent 2 used in the previous episodes) and vice-versa; this behavior has been already documented in [4]. We can try to counteract this problem by implementing a "freezing window". This way we update only one agent after each episode, and only after a certain number of training sessions of agent 1 we then switch to agent 2 and vice-versa. Alas only this measure was not sufficient to counteract the effect of this non-stationary target.

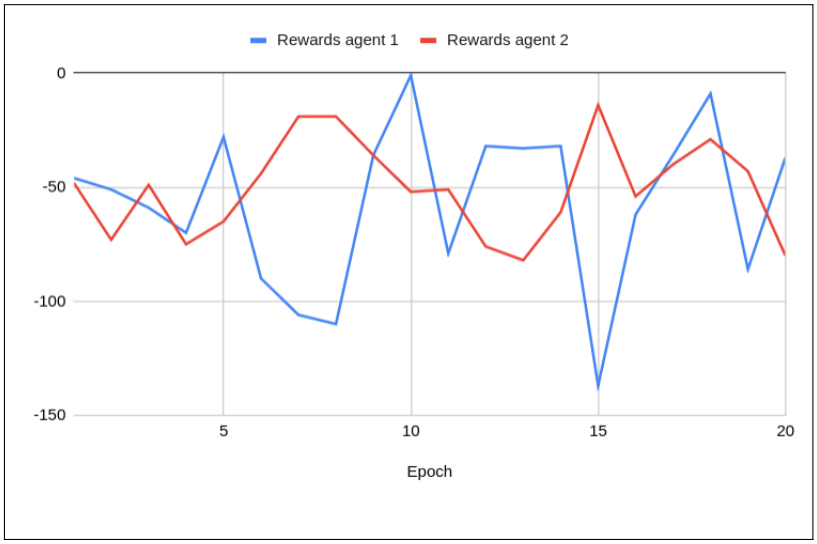


Figure 2: training with the "freezing window"

2.2 Refining and improving the algorithm

Introducing entropy Aside from the non-stationary target, in the previous training sessions there seems to be a problem of insufficient exploration. Since the positive rewards of both agents depend on collaborating, they both land on a strategy of no cooperation and no defection, that is: stay away from the stag (since if caught they would receive a negative reward) and occasionally land on some plants to receive a small positive reward.

We can try to improve this by including entropy to the policy loss to discourage deterministic policies. This should offer more possibilities to earn the positive reward. We define entropy as:

$$E_{\pi(a|s)} = -p(a|s)ln(a|s) \tag{11}$$

We can then compute the final policy loss as:

$$J_T = J(\theta) - \gamma E_{\pi(a|s)} \tag{12}$$

with γ as the entropy coefficient, an hyperparameter that will regulate how much the agent will be incentivize to differ from the learned policy. Even better, we can modulate this parameter so that if the number of cooperative actions (successful hunts) increased from the last training session.

Baseline minibatch In order to better understand the limitation of the previous version of the algorithm, we can look at its explained variance. The definition of Explained variance is:

$$EV = 1 - \frac{\sigma^2(G_t - MSE(G_t - \hat{v}(S_t, \omega)))}{\sigma^2(G_t)} \tag{13}$$

a low but positive value of explained variance can be caused by too few gradient updates, so we can eliminate this problem by creating a mechanism for minibatch gradient updates: when training the

baseline network, we divide the states and discounted returns tensors in many batches and we apply Stochastic Gradient Descent on random samples from each minibatch. We cannot use this method also on the agent network because updating the policy and then re-updating it on a batch of steps obtained from a different policy will effect negatively the training.

Reward shaping There is ample evidence that games with sparse rewards can cause additional difficulties when training an agent [5]. In our case, the agents obtain a reward only when arriving on a cell with a stag or a plant, but they do not receive any reward otherwise. We can solve this by adding a component in the reward of each that keeps track of the distance between the agent and the stag and the two agents themselves, since the cooperation requires also coordination. There is a problem with reward shaping: it can lead to unwanted behaviors. For example, by simply subtracting the distances from the stag from the reward, the agents could find that it would be sufficient to "follow closely" the stag, and not commit to the hunt(this is documented in [6]). Instead of using a simple shaping function, we will use instead potential-based shaping:

$$G'_t = G_t + F(s, a, s') = G_t + \delta(\gamma\Phi(s') - \Phi(s)) \tag{14}$$

In our case, γ will be a discount factor with $\gamma \in [0, 1)$, $\Phi(s)$ and $\Phi(s')$ will be the euclidean distances between both actors and the stag, δ will be a scaling factor used to prevent the value of the distance reward to overtake the environment rewards.

2.3 Proximal Policy Optimization (PPO)

Since we were not able to solve the environment using Reinforce, we can try to use Proximal Policy Optimization (PPO) [7]. PPO has many advantages over Reinforce, two that can prove useful are: it allows several gradient updates on the same batch of data, while explicitly discouraging updates that change the policy too much.

Implementing advantage Choosing $b(S_t) = \hat{v}(S_t, w)$, the term multiplying $\nabla \ln \pi$ in the update is $G_t - \hat{v}(S_t, w)$. Since $E_\pi[G_t \mid S_t = s, A_t = a] = q_\pi(s, a)$, this is a sample estimate (with the sampled return G_t in place of $\hat{Q}$) of the advantage function

$$\delta_{ADV}(s, a) = \hat{Q}(s, a, w) - \hat{v}(s, w) \tag{15}$$

which measures how much better (or worse) action a is than the average behavior of the policy in state s. The update of Reinforce with baseline can therefore be written as

$$\theta_{t+1} = \theta_t + \alpha\, \delta_{ADV}\, \nabla \ln \pi(A_t|S_t, \theta_t) \tag{16}$$

Reinforce can use each batch for a single update only: this update is a faithful estimate of $\nabla J(\theta)$ only if the samples were generated by the very same θ_t we are differentiating, so after one step the data is outdated, and further steps on it can move θ too far and destroy the policy's performance. PPO allows multiple updates on the same batch, while keeping the new policy close to the one that generated the data.

Reusing data Through importance sampling, data collected with the current policy π_{θ_t} can be reused to evaluate a different policy π_θ by weighting each sample with the ratio $\pi_\theta(a|s)/\pi_{\theta_t}(a|s)$, which equals 1 for $\theta = \theta_t$ and is greater (smaller) than 1 if the action a has become more (less) likely under the new policy. Replacing the logarithm with this ratio gives the surrogate advantage

$$\mathcal{L}(\theta_t, \theta) = E_{s,a\sim\pi_{\theta_t}}\left[\frac{\pi_\theta(a|s)}{\pi_{\theta_t}(a|s)}\, \delta^{\pi_{\theta_t}}_{ADV}(s, a)\right] \tag{17}$$

which measures how the new policy π_θ performs relative to the old policy π_{θ_t}, using data from the old policy. Since $\nabla\pi/\pi = \nabla \ln \pi$, its gradient at $\theta = \theta_t$ is

$$\nabla\mathcal{L}(\theta_t, \theta)\Big|_{\theta=\theta_t} = E_{s,a\sim\pi_{\theta_t}}\left[\delta_{ADV}\, \nabla \ln \pi(a|s, \theta_t)\right] \tag{18}$$

exactly the Reinforce with baseline gradient. However, $\mathcal{L}$ is reliable only while π_θ stays close to π_{θ_t}, and maximizing it without constraints would lead again to excessively large updates.

The clipped objective (PPO-CLIP) Trust Region Policy Optimization (TRPO)[8] enforces this closeness by maximizing $\mathcal{L}(\theta_t, \theta)$ under an explicit constraint on the KL divergence between the old and the new policy, $D_{KL}(\theta||\theta_t) \leq const$, but this is very complex to implement. PPO obtains a similar behavior with a first-order optimization method, by modifying the objective so that moving the ratio too far from 1 brings no additional benefit. The update is

$$\theta_{t+1} = \arg\max_{\theta} E_{s,a\sim\pi_{\theta_t}} \left[L(s, a, \theta_t, \theta)\right] \tag{19}$$

with

$$L(s, a, \theta_t, \theta) = \min\left(\frac{\pi_\theta(a|s)}{\pi_{\theta_t}(a|s)}\,\delta_{ADV},\ \text{clip}\Big(\frac{\pi_\theta(a|s)}{\pi_{\theta_t}(a|s)}, 1-\epsilon, 1+\epsilon\Big)\delta_{ADV}\right) \tag{20}$$

where $\delta_{ADV} = \delta^{\pi_{\theta_t}}_{ADV}(s, a)$, $\text{clip}(x, a, b) = \min(\max(x, a), b)$ and ϵ is a hyperparameter that quantifies how far the new policy is allowed to move from the old one. If $\delta_{ADV} \geq 0$ the term equals $\min\big(\pi_\theta(a|s)/\pi_{\theta_t}(a|s),\ 1+\epsilon\big)\delta_{ADV}$: it hits a ceiling of $(1+\epsilon)\delta_{ADV}$ as soon as $\pi_\theta(a|s) > (1+\epsilon)\pi_{\theta_t}(a|s)$, so the action is not made any more likely. If $\delta_{ADV} < 0$ it equals $\max\big(\pi_\theta(a|s)/\pi_{\theta_t}(a|s),\ 1-\epsilon\big)\delta_{ADV}$, with a ceiling of $(1-\epsilon)\delta_{ADV}$ once $\pi_\theta(a|s) < (1-\epsilon)\pi_{\theta_t}(a|s)$. The min removes the incentive to change the policy only in the direction that improves the objective: if the ratio has moved in the wrong direction, the term is not clipped and the full gradient corrects the mistake. This is not a hard constraint, but with a small learning rate and few epochs per batch the new policy stays in the neighborhood of the old one.

Final objective and algorithm The baseline $\hat{v}$ is trained by regression towards the return, with loss $L^{VF}(s, w) = (\hat{v}(s, w) - G_t)^2$, and an entropy bonus $H(\pi_\theta(\cdot|s)) = -\sum_a \pi_\theta(a|s) \ln \pi_\theta(a|s)$ is usually added to preserve exploration. The complete PPO objective is

$$L^{PPO}(\theta, w) = E_{s,a\sim\pi_{\theta_t}}\left[L(s, a, \theta_t, \theta) - c_1\, L^{VF}(s, w) + c_2\, \mathcal{H}(\pi_\theta(\cdot|s))\right] \tag{21}$$

with c_1, c_2 weighting coefficients; as before, backpropagation minimizes $-L^{PPO}$. Each iteration of PPO then consists of: collecting a batch of experience with the current policy π_{θ_t}, computing G_t and δ_{ADV} for each step and storing the values $\ln \pi_{\theta_t}(a|s)$; performing K epochs of mini-batch stochastic gradient steps on $-L^{PPO}$, computing the ratio as $\exp\big(\ln \pi_\theta(a|s) - \ln \pi_{\theta_t}(a|s)\big)$; taking the resulting parameters as θ_{t+1}, discarding the batch and repeating.

3 Results and Future developments

Regarding the Reinforce with baseline algorithm, the improvements applied were not sufficient to stabilize learning and even allow the agents to cooperate. Figures 3 and 4[1] show a rare instance where the agents were able to learn to cooperate, but this is quite rare and usually the number of cooperative events remain in $[0, 6]$ and the rewards rarely become positive for a given epoch, due to the high number of defections.

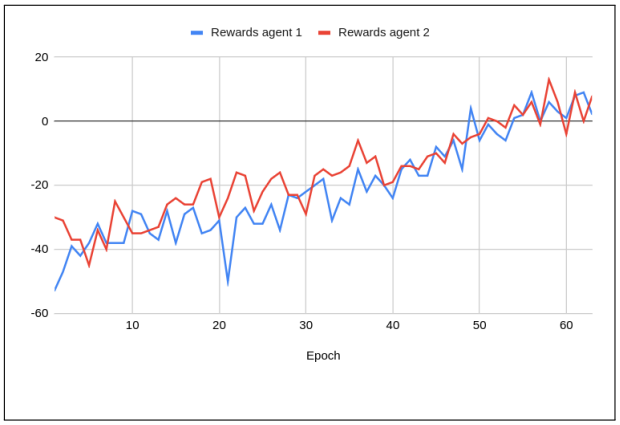


Figure 3: Rewards in the successful case

[1] regarding the last graph, results from epoch 1-23 are not included since the number of cooperative actions was negligible, and I wanted to emphasize the intervals where cooperative actions start to increase while defections decrease.

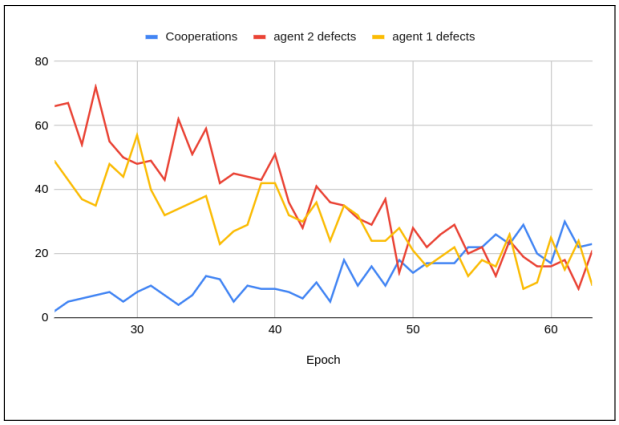


Figure 4: number of cooperative actions and defections in the successful case

Even PPO was not able to solve StagHunt, in this case we can see a lower number of both defections and cooperations, the total rewards are overall higher than Reinforce because of the lower number of defections. I also noted that in both cases the Explained Variance remains approximately in $[-1.5, 0.1]$, these values indicates that the baseline model was not able to predict the rewards of the next training, initially I thought it was caused by the sparsity of the rewards, but adding the shaping factor did not improved it.

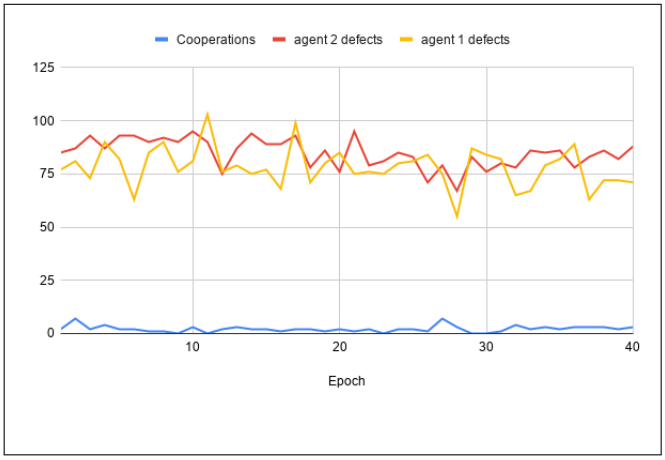


Figure 5: number of cooperative actions and defections in the unsuccessful case, Reinforce with baseline

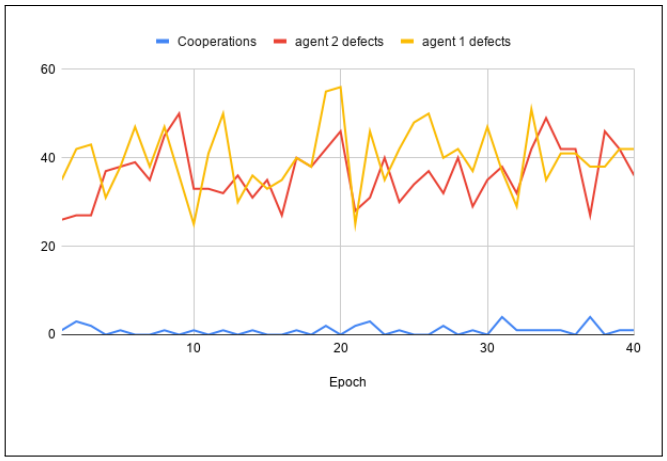


Figure 6: number of cooperative actions and defections in the unsuccessful case, PPO

Future attempts should be aimed at trying to implement Multi Agent PPO (MAPPO), where we use a centralized critic to distinguish more clearly cooperation from defection[7]. Also attention should be posed on understanding what causes the Explained Variance to remain unchanged.

4 Reference

[1] Sutton, R. S., & Barto, A. G. (2018). Reinforcement Learning: An Introduction (2nd ed.). MIT Press.

[2] Brewka G. Artificial intelligence—a modern approach by Stuart Russell and Peter Norvig, Prentice Hall. Series in Artificial Intelligence, Englewood Cliffs, NJ. The Knowledge Engineering Review. 1996;11(1):78-79. doi:10.1017/S0269888900007724

[3] Hasselmo, M.E., Schnell, E. & Barkai, E. (1995) Dynamics of learning and recall at excitatory recurrent synapses and cholinergic modulation in rat hippocampal region CA3. *Journal of Neuroscience* **15**(7):5249-5262.

[4] Tuyls, K. and Weiss, G. (2012), Multiagent Learning: Basics, Challenges, and Prospects. AI Magazine, 33: 41-52. https://doi.org/10.1609/aimag.v33i3.2426

[5] Ionel-Alexandru Hosu and Traian Rebedea (2016), Playing Atari Games with Deep Reinforcement Learning and Human Checkpoint Replay, https://arxiv.org/abs/1607.05077

[6] Ng, Andrew Y., Daishi Harada, and Stuart Russell. "Policy invariance under reward transformations: Theory and application to reward shaping." Icml. Vol. 99. 1999.

[7] YU, Chao, et al. The surprising effectiveness of ppo in cooperative multi-agent games. Advances in neural information processing systems, 2022, 35: 24611-24624.

[8] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, O. Klimov, "Proximal Policy Optimization Algorithms", arXiv:1707.06347, 2017.